

## Real-world patterns I keep seeing

A few patterns come up over and over in real projects.

*Semantic search over internal documentation:* Probably the most common entry point. Take all your wikis, runbooks, support articles, whatever. Chunk them into pieces of a few hundred words each. Embed each chunk. Store the embeddings. When a user asks a question, embed the question, find the closest chunks, optionally show them as snippets and links. This alone can replace a surprising amount of manual searching.

*Retrieval-augmented chatbots:* Same as the above, but instead of just showing results, you feed the retrieved chunks into an LLM along with the user's question and let the model write a natural answer. Most 'AI assistant' products you've used in the last year work roughly like this under the hood.

*Product recommendations and similarity search:* E-commerce sites use embeddings of product descriptions, images, and user behaviour to surface similar items. Vector databases make this fast even with millions of products.

*Duplicate detection and deduplication:* Got a support ticket queue with lots of repeats? Embed each ticket. Find clusters in the vector space. Now you have automatic deduplication that works even when the wording is completely different.

*Image and audio search:* Embeddings aren't just for text. CLIP-style models produce embeddings of images that capture their meaning. You can search a million images by visual concept rather than by file name. Similar tools exist for audio.

The thing I find interesting is how often these projects start small and grow. Someone builds a quick prototype with Chroma to search a few hundred documents. Six months later it's serving a thousand employees, and they're moving to Qdrant or pgvector for production. The journey from 'weekend

hack' to 'core product feature' is shorter for vector search than for almost any other AI capability.

## Embedding choices matter more than database choices

Something I see developers underestimate is how much the choice of embedding model affects the quality of search results.

The same query against the same documents can produce dramatically different results depending on which model you used to generate the embeddings. A model trained on general English will work fine for most things but may struggle with technical jargon. A model trained specifically on code will give you great results for code search but mediocre results for prose. A multilingual model handles many languages but might be slightly less accurate on any single one than a language-specific model.

## A few things to consider when picking a model

Dimension size matters for storage and speed. Smaller embeddings, like 384 or 512 dimensions, are faster to store and search. Larger embeddings, like 1024 or 1536 dimensions, can capture more nuance but cost more. For most applications, smaller is fine.

Domain matches matter a lot. If you're searching legal documents, look for a model fine-tuned on legal text. If you're doing code search, models like CodeBERT or all-mpnet might serve you better than generic ones.

Multilingual support matters if your users are international. The BGE-M3 and multilingual-e5 models are great options here.

For most general purposes, all-MiniLM-L6-v2 from sentence-transformers is a solid starting point. It produces 384-dimensional embeddings, runs on a CPU, and is genuinely good for general English text. Start there. Switch later if you need something more specialised.

## Things that bite people

I've made and watched others make a fair few mistakes with vector databases. A handful keep coming up.

*Forgetting to normalise vectors:* Some distance metrics, like cosine similarity, work best when vectors are normalised to unit length. Some embedding models output normalised vectors. Some don't. Check your model's documentation and your database's expectations. Mismatches can silently produce bad results.

*Bad chunking strategies:* When you're embedding documents, how you split them up matters enormously. Chunks that are too small lose context. Chunks that are too large dilute the signal. Most teams settle on chunks of around 200 to 500 tokens with some overlap, but this is worth experimenting with for your specific use case.

*Ignoring the underlying metric:* Most vector databases support multiple distance metrics. Cosine similarity is the most common for text embeddings, but Euclidean distance and inner product are also options. Picking the wrong one for your model can hurt result quality. When in doubt, check what the embedding model's authors recommend.

*Treating vector search as a replacement for keyword search:* It isn't. Vector search is great at finding semantically similar content. Keyword search is great at finding exact terms. The best systems combine both, often called hybrid search. If a user searches for an exact error code, keyword search will find it. If they search for the meaning behind it, vector search will. Use both where possible.

*Not tracking embedding model versions:* If you upgrade your embedding model, your old vectors are essentially garbage. They were generated by a different function. You need to re-embed your entire corpus when you change models. Plan for this and budget for it.